

4.1 - Criptografia simétrica e assimétrica

O arquiteto decide o uso, não implementa o algoritmo

CNHs e dados bancários em texto simples

- **LunaLog precisa guardar CNHs e dados bancários de motoristas**
 - Dados usados para processar pagamentos recorrentes

LUNALOG

Em 2022, a LunaLog passou a processar pagamentos de motoristas e precisou armazenar CNHs digitalizadas e dados bancários. O desenvolvedor responsável perguntou a Roberto como guardar aquilo. Roberto olhou o código existente e respondeu que estava tudo em texto simples no banco, igual ao resto do sistema. Ninguém na sala sabia explicar a diferença entre proteger aqueles dados e apenas confiar que o banco jamais vazaria. Essa era exatamente a decisão que Roberto não estava preparado para tomar.

Três primitivas que todo arquiteto domina

Não para implementar, mas para decidir o uso

- **Criptografia simétrica usa uma única chave compartilhada**
 - AES é o padrão atual, rápido e eficiente para grandes volumes
- **Criptografia assimétrica usa par de chaves pública e privada**
 - RSA e curvas elípticas resolvem a distribuição de chave
- **Hashing gera impressão digital irreversível do dado**
 - SHA-256 e bcrypt não permitem recuperar o original
- **Arquiteto escolhe a primitiva certa para cada problema**
 - Errar a primitiva quebra a segurança independentemente da implementação

Simétrica e assimétrica lado a lado

Simétrica (AES)

- Uma chave para cifrar e decifrar
- Altíssimo desempenho em grandes volumes
- Problema: distribuir a chave com segurança
- Ideal para dados em repouso e payloads

Assimétrica (RSA, ECC)

- Par de chaves, pública e privada
- Desempenho menor, custo computacional alto
- Resolve distribuição: a chave pública é aberta
- Ideal para troca de chave e assinatura

Por que protocolos reais combinam as duas

- TLS usa assimétrica só para negociar a chave de sessão
- A partir daí a sessão inteira roda em simétrica

A assimétrica resolve a distribuição da chave, a simétrica resolve a velocidade. Nenhum protocolo sério escolhe uma só, ele combina as duas.

- Esse padrão híbrido é a base de TLS, SSH e PGP

Hashing: quando o objetivo é não reverter

- **Hash valida integridade sem expor o dado original**
 - Comparar hashes confirma que nada foi alterado
- **Senha nunca é cifrada, é sempre hasheada com salt**
 - bcrypt, scrypt e Argon2 são lentos de propósito
- **Hash criptográfico é determinístico e resistente a colisão**
 - MD5 e SHA-1 estão quebrados, não use em contexto novo
- **Confundir hash com criptografia gera falha de design**



Você já sabe qual primitiva usar em cada situação. Mas existe um detalhe que derruba tudo: criptografia que protege o dado parado não vale nada se ele trafega aberto pela rede entre dois serviços. Onde exatamente a proteção precisa estar quando os dados estão em movimento?

TLS e mTLS em sistemas distribuídos